Ruh.ai V1 Employee Pack Specification

Reusable packaging standard for governed digital employees built on OpenClaw gateway runtimes

Document type	Version	Status	Date
System contract	1.0	Working draft	7 March 2026

What this document does. It defines the reusable pack format, authoring rules, collaboration contracts, governance model, validation gates, and deployment lifecycle used to create digital employees that can work together across many customers without forking the product.

Companion documents. Use this specification together with the platform PRD, the V1 System Design & MVP Execution Blueprint, and the V1 API & Event Contract Specification. The pack spec is the bridge between architecture and deployable employees.

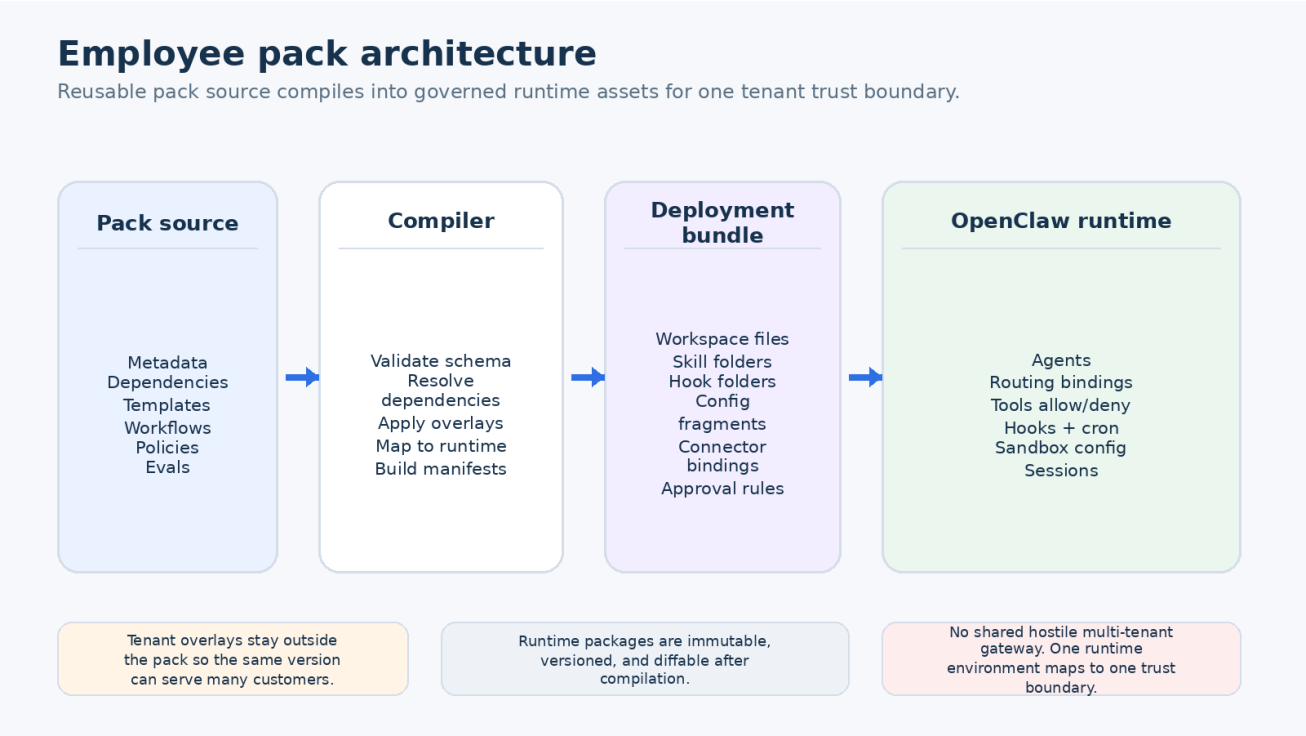

Figure 1. The employee pack is the reusable source artifact; tenant overlays and runtime bundles stay separate.

Executive snapshot

A pack is the highest-level reusable product artifact in Ruh.ai. It is the unit that product, solution engineering, and platform engineering can author once and deploy many times. A pack does not represent a single tenant deployment. Instead it packages customer-neutral definitions for employee roles, workflow contracts, artifact schemas, approval logic, connector mappings, and eval suites.

The central design decision is that packs remain declarative by default and customer-specific values live in overlays. That gives Ruh.ai a way to serve multiple construction companies, software teams, and other future industries without cloning prompts or maintaining one-off agent workspaces for each customer.

- Packs must compile deterministically into runtime bundles for one trust boundary at a time.
- Packs must support cooperating employees through typed tasks, shared artifacts, locks, and approvals - not uncontrolled cross-chat.
- Packs must be promotable through dev, staging, and production without source edits.
- Pack releases are immutable, versioned, signed, and observable after deployment.

1. Purpose, scope, and non-goals

This document defines the build contract for Ruh.ai employee packs. It specifies what a pack is, how it is structured, how it maps to OpenClaw runtime assets, what may vary by tenant and environment, how employees inside a pack collaborate, and what validation a pack must pass before release.

The target audience is the platform team, runtime team, solution engineers, pack authors, and design-partner implementation leads. A pack author should be able to build or update a reusable industry solution without inventing new directory structures, schema names, or deployment conventions.

1.1 This document is not

- It is not a customer proposal or sales deck.
- It is not the low-level REST or event schema; those live in the API & Event Contract Specification.
- It is not a promise that every future pack feature ships in V1. Anything not marked v1-safe is aspirational.
- It is not a permission slip for arbitrary code inside production packs.

2. Design constraints inherited from OpenClaw

The pack system exists above OpenClaw, but it must respect OpenClaw's runtime model. OpenClaw already provides isolated agents with workspaces, auth, routing bindings, typed tools, hooks, and cron scheduling [R1][R2][R3][R4]. OpenClaw also expects workspace bootstrap files such as AGENTS.md, SOUL.md, USER.md, IDENTITY.md, TOOLS.md, HEARTBEAT.md, BOOT.md, and BOOTSTRAP.md [R5].

The most important boundary condition is security. OpenClaw explicitly documents a personal-assistant trust model: one trusted operator boundary per gateway, not a hostile multi-tenant shared

bus [R6]. Its workspace is the default working directory but not a hard sandbox unless sandboxing is enabled [R5]. Sandboxing is optional, Docker-based, and configurable by mode, scope, and workspace access [R7][R8].

The pack specification therefore assumes that Ruh.ai supplies the enterprise control plane and uses OpenClaw as a trust-bounded runtime operating layer. Every compiled deployment targets one runtime environment for one tenant trust boundary.

OpenClaw fact	Why the pack spec must care
Agents manage isolated workspaces, auth, and routing bindings [R1].	A template must compile into a clear agent identity, workspace tree, and channel routing plan.
Skills are directory-based, AgentSkills-compatible folders with precedence across bundled, managed, and workspace locations [R2].	Pack authors need a standard way to ship reusable skill assets without relying on ad hoc prompt files.
Hooks are discovered from workspace, managed, and bundled directories [R3].	Pack automation must declare where hook behavior comes from and whether it is safe for production promotion.
Tool access can be allow/deny controlled globally and per agent [R4].	Every employee template needs an explicit tool profile instead of relying on default tool exposure.
Workspace files are injected into prompt context and large files are truncated beyond configured limits [R5][R9].	Prompt assets inside a pack require size budgets and a compile-time lint step.
OpenClaw supports optional sandboxing with workspace access modes none, ro, and rw [R7][R8].	Each employee template must declare its minimum sandbox posture and whether host workspace access is required.

3. Why employee packs exist

Without a pack abstraction, every customer deployment becomes a bespoke collection of prompts, connector secrets, cron jobs, and runtime tweaks. That approach does not scale across the construction industry or any other industry because knowledge stays trapped inside one-off workspaces and field engineers become the integration layer.

Packs solve that by separating four concerns that should not be mixed together: reusable business logic, customer-specific configuration, compiled runtime assets, and live operational data.

- Reusable business logic belongs in the pack source: templates, workflows, artifact schemas, eval suites, policies, and connector mappings.
- Customer-specific values belong in overlays: project keys, distribution lists, allowed channels, branding strings, approval groups, and connector installation refs.
- Compiled runtime assets belong in versioned deployment bundles: workspace files, skill folders, hook refs, routing bindings, and config fragments.
- Operational state belongs in Ruh.ai resources and logs: work items, tasks, artifacts, approvals, run telemetry, and outcomes.

Design rule. A pack release must stay customer-neutral. If a solution cannot be deployed to a second customer without editing the pack source, it is not a valid pack - it is a bespoke implementation.

4. Canonical pack model

A pack is an immutable, versioned distribution unit. It may depend on other packs, export one or more employee templates and workflows, and declare the policies, schemas, assets, and evals required to deploy those exports safely.

4.1 Pack types

Pack type is a classification field, not a security boundary. V1 supports three types.

Pack type	Intent	Typical contents
foundation	Cross-industry primitives shared by many solutions.	Common artifact schemas, generic employee templates, baseline policies, utility skills, and shared eval patterns.
domain	Industry or functional specialization built on top of foundation packs.	Construction operations workflows, document-control templates, or BuildOps templates for AI product teams.
solution	Opinionated composition intended for fast deployment.	Selected templates, starter workflows, dashboards, default policies, and connector profiles designed for a particular solution motion.

4.2 Canonical objects inside a pack

Object	What it defines	Cardinality in one pack
Pack manifest	Identity, compatibility, dependencies, exports, and allowed overlays.	Exactly one
Employee template	A reusable role contract that can compile into a runtime employee.	One or more
Workflow template	State machine, task graph, assignee rules, and artifact expectations.	Zero or more
Task contract	Typed input and output schema for a unit of work.	Zero or more
Artifact schema	Persisted business object produced or consumed by workflows.	Zero or more

Object	What it defines	Cardinality in one pack
Policy profile	Approval thresholds, tool restrictions, sandbox posture, and write-back rules.	Zero or more
Connector profile	Normalized capabilities mapped to provider-specific operations.	Zero or more
Eval suite	Quality gates, test fixtures, scoring rubrics, and pass thresholds.	Zero or more
Asset bundle	Prompt fragments, skill folders, reference docs, images, and examples.	Zero or more

5. Mapping pack objects to OpenClaw runtime assets

Packs do not run directly. The deployment compiler converts them into runtime bundles that OpenClaw can execute. The mapping must be explicit and deterministic so that pack authors do not need to hand-edit gateway files during deployment.

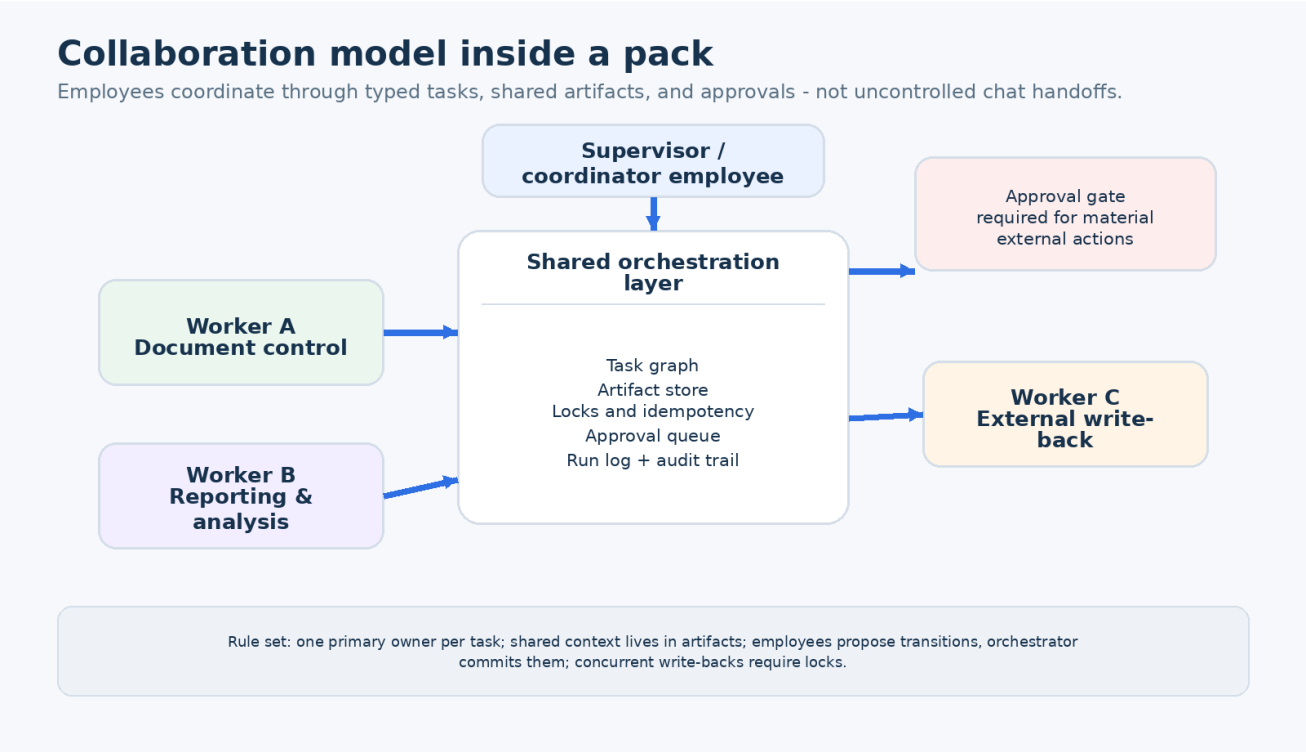

Figure 2. Recommended collaboration pattern inside a pack: supervisor coordinates workers through shared orchestration resources.

Ruh.ai pack object	Compiled OpenClaw target	Notes
Employee template	agents.list[] entry + workspace root + routing bindings	Includes identity, workspace path, tool profile, model profile, and channel routing plan.

Ruh.ai pack object	Compiled OpenClaw target	Notes
Prompt assets	Workspace files such as AGENTS.md, SOUL.md, USER.md, IDENTITY.md, TOOLS.md, HEARTBEAT.md, BOOT.md	Compiler controls file size and injection order.
Skill asset	workspace/skills/<skill>/SKILL.md or managed skill directory	Use workspace-level install by default for reproducibility; managed install only for signed shared assets.
Hook reference	workspace/hooks/<hook> or managed hook package	Raw hook code is restricted in certified releases; curated package refs are preferred.
Tool profile	tools.allow / tools.deny and per-agent tool config	Deny-by-default on high-risk tools for external-facing employees.
Connector profile	Plugin config + connector binding manifest + capability map	Secrets are injected by reference at deployment time, never committed to the pack.
Heartbeat plan	HEARTBEAT.md and/or cron jobs	Use heartbeat for lightweight recurring review and cron for explicit schedules [R10].
Sandbox profile	agents.defaults.sandbox or agents.list[].sandbox	Pack declares minimum required posture; runtime may tighten it further.

6. Pack directory and file structure

Every pack release resolves to a canonical source tree. The source tree must be small enough for code review, structured enough for tooling, and strict enough to prevent pack authors from hiding important behavior in untracked files.

6.1 Required root files

Path	Required	Purpose
pack.yaml	Yes	Primary manifest for identity, compatibility, dependencies, exports, overlay policy, and release metadata.
README.md	Yes	Human-readable overview, pack intent, included templates, and deployment notes.
CHANGELOG.md	Yes	Release history and upgrade notes.
LICENSE.md	Optional	Needed if distribution rules differ by pack source.

Path	Required	Purpose
examples/	Optional	Sample overlays, test payloads, and walkthrough scenarios.

6.2 Standard directories

Directory	Contents	V1 rule
templates/	Employee template manifests and prompt assets.	Required if the pack exports employee templates.
workflows/	Workflow graphs, state machines, and task contracts.	Required if the pack exports workflows.
artifacts/	Artifact schemas, render configs, and document mappings.	Recommended for any pack that persists business state.
policies/	Approval, sandbox, tool, and risk profiles.	Required for any template that can produce side effects.
connectors/	Capability maps, connector presets, and binding schemas.	Required when the pack integrates with external systems.
skills/	Reusable skill folders shipped with the pack.	Optional, but preferred over long AGENTS.md files for tool guidance [R2].
evals/	Fixtures, scoring rubrics, thresholds, and regression suites.	Required for certified releases.
assets/	Images, reference docs, UI metadata, and pack diagrams.	Optional.
mappings/	Compiler mapping files from pack objects to runtime outputs.	Required when defaults are not enough.
migrations/	Upgrade or data migration descriptors between pack versions.	Optional in V1, required once breaking data changes appear.

```
pack/
  pack.yaml
  README.md
  CHANGELOG.md
  templates/
    triage/
      template.yaml
      prompts/
        AGENTS.md
        SOUL.md
        TOOLS.md
        HEARTBEAT.md
  workflows/
    issue_to_resolution.workflow.yaml
    contracts/
      intake_task.schema.json
  artifacts/
    triage_packet.schema.json
    summary_report.schema.json
  policies/
    tool_profiles/
    approval_profiles/
    sandbox_profiles/
  connectors/
    github.binding.yaml
    slack.binding.yaml
  skills/
    evidence_summarization/
      SKILL.md
  evals/
    core/
      smoke.yaml
      regression.yaml
  assets/
    icons/
    docs/
  mappings/
    runtime_map.yaml
```

6.3 Executable content rules

V1 prefers declarative packs. Markdown, YAML, JSON, images, and static reference files are allowed by default. Executable behavior is much more restricted because OpenClaw plugins and host-side extensions live inside the runtime trust boundary [R6].

- Certified production packs may not embed raw secrets, API keys, OAuth tokens, or customer IDs.
- Certified production packs may not rely on arbitrary shell scripts or unreviewed binaries.
- Hook or plugin behavior should be referenced through curated package IDs and pinned versions. Inline source is allowed only in internal developer mode and must be converted to signed package refs before promotion.
- The publish pipeline must scan the pack tree for executable files, unknown file types, and unresolved external references.

7. Pack manifest specification

The root manifest is the pack's source of truth. It must declare what the pack exports, what it depends on, which overlays are allowed, and which platform and runtime versions are compatible. Nothing outside the manifest may silently change the identity of a pack release.

7.1 Required manifest fields

Field	Type	Meaning
apiVersion	string	Schema version for the pack format, e.g. ruh.ai/v1alpha1.
kind	enum	Always EmployeePack for this document.
metadata.id	string	Globally unique pack identifier.
metadata.version	semver	Immutable released version for the pack.
metadata.title	string	Human-readable pack name shown in studio and registry.
metadata.packType	enum	foundation | domain | solution.
compatibility.platform	range	Supported Ruh.ai platform versions.
compatibility.runtimeAdapter	range	Supported Ruh runtime adapter versions.
compatibility.openclaw	range	Supported OpenClaw runtime range used by certified bundles.
dependencies[]	list	Other packs or shared assets required by this pack.
exports	object	Templates, workflows, artifact schemas, and presets exposed to downstream users.
overlays.allowed[]	list	Field paths that tenants or environments may override without forking the pack.
security	object	Executable-content mode, default sandbox posture, and release hardening requirements.

```yaml
apiVersion: ruh.ai/v1alpha1
kind: EmployeePack
metadata:
  id: ai-buildops
  version: 1.2.0
  title: AI BuildOps Pack
  packType: solution
  owner: ruh-core
  summary: >
    Digital employees for intake, specification, build orchestration,
    evals, and release preparation.
compatibility:
  platform: ">=1.0.0 <2.0.0"
  runtimeAdapter: ">=1.0.0 <2.0.0"
  openclaw: ">=2026.2 <2027.0"
dependencies:
  - id: foundation-core
    version: "^1.4.0"
exports:
  templates:
    - buildops.triage
    - buildops.spec
    - buildops.release
  workflows:
    - issue_to_pull_request
overlays:
  allowed:
    - vars.brand.displayName
    - vars.sla.defaultHours
    - connectors.github.installationRef
    - connectors.jira.projectKey
    - policies.externalWriteback.approverGroup
security:
  executableContent: curated_code_refs_only
  defaultSandboxMode: all
  defaultWorkspaceAccess: none
```

7.2 Overlay rules

Overlay policy is the mechanism that keeps packs reusable. A pack release must explicitly list which fields may vary after publish. Everything else is immutable. That prevents silent drift between customers and ensures that release quality remains meaningful.

- Allowed overlays should focus on values, not behavior: names, project keys, channel ids, approval groups, connector installation refs, distribution lists, and SLA targets.
- Behavioral changes such as workflow topology, tool exposure, or risk tier should require a new pack version unless the manifest explicitly marks a narrow override point as safe.
- Overlay precedence from lowest to highest is: dependency defaults → pack defaults → solution preset → tenant overlay → environment overlay → emergency deployment flags.
- Only fields marked overridable in the manifest may participate in precedence resolution.

8. Employee template specification

An employee template is a customer-neutral role contract. It describes what the employee is for, what work it can accept, what outputs it must produce, what context it needs, what tools and capabilities it is allowed to use, what approval rules govern it, and how it collaborates with other employees.

8.1 Required template fields

Field group	Required fields	Intent
metadata	id, title, summary, owner	Identity and display metadata.
role	objective, businessOutcome	What success looks like for this employee.
contracts	accepts, emits	Signal, task, and artifact contracts the employee consumes and produces.
context	workspaceFiles, skills, memoryProfileRef	What instructions, local notes, and memory policies are injected.
tools	toolProfileRef, requiredCapabilities	Normalized capability need and tool exposure policy.
governance	policyProfileRef, riskTier	Approval and side-effect controls.
execution	modelProfileRef, sandboxProfileRef	Runtime model selection and isolation posture.
collaboration	delegatesTo, supervisorEligible	How the employee participates in teamwork.
evals	suiteRefs	Quality gates required before promotion.

```yaml
apiVersion: ruh.ai/v1alpha1
kind: EmployeeTemplate
metadata:
  id: buildops.triage
  title: Intake & Triage Employee
  summary: Normalize inbound work into typed work items.
role:
  objective: >
    Convert messy incoming signals into a clean work item with severity,
    route, evidence links, and next-step recommendation.
contracts:
  accepts:
    - signal.issue_reported
    - signal.alert_opened
    - signal.customer_feedback
  emits:
    - work_item.normalized
    - artifact.triage_packet
context:
  workspaceFiles:
    agents: prompts/triage/AGENTS.md
    soul: prompts/common/SOUL.md
    tools: prompts/common/TOOLS.md
    heartbeat: prompts/triage/HEARTBEAT.md
  skills:
    - skills/evidence_summarization
    - skills/issue_classification
  memoryProfileRef: policies/memory/short_context.yaml
tools:
  toolProfileRef: policies/tool_profiles/triage_lowrisk.yaml
  requiredCapabilities:
    - messages.read
    - files.read
    - issues.create
    - artifacts.write
governance:
  policyProfileRef: policies/approval_profiles/triage.yaml
  riskTier: L1
execution:
  modelProfileRef: models/reasoner_medium.yaml
  sandboxProfileRef: policies/sandbox/all_none.yaml
collaboration:
  supervisorEligible: true
  delegatesTo:
    - buildops.spec
evals:
  suiteRefs:
    - evals/core/triage_smoke.yaml
    - evals/core/triage_regression.yaml
```

8.2 Workspace and prompt asset mapping

OpenClaw loads operating instructions from workspace files and may also read skills on demand [R2] [R5]. Pack authors should therefore split long, reusable tool guidance into skills and keep workspace files concise, specific, and bounded by purpose.

Workspace asset	What it should contain	Pack authoring rule
AGENTS.md	Role instructions, priorities, work norms, memory behavior, and decision policy.	Keep role-specific and concise; avoid copying connector specifics that belong in TOOLS.md or skills.

Workspace asset	What it should contain	Pack authoring rule
SOUL.md	Tone, persona, boundaries, and interaction style.	Small and stable across many templates where possible.
USER.md	How the employee should address operators or end users.	Usually populated from overlays, not hard-coded.
IDENTITY.md	Name, vibe, avatar metadata.	Brandable through overlays; never used for core business logic.
TOOLS.md	Local conventions, aliases, and environment-specific notes.	Prefer tenant overlays or connector metadata instead of free-form customer prose.
HEARTBEAT.md	Short recurring review checklist.	Keep tiny to reduce token burn [R10].
BOOT.md / BOOTSTRAP.md	Startup or first-run rituals when explicitly needed.	Avoid pack dependence on one-time human rituals in customer production environments.
skills/*/SKILL.md	Reusable tool usage guidance and step-specific instructions.	Preferred place for reusable methods and procedures [R2].

8.3 Prompt and context budgets

Because OpenClaw injects workspace and bootstrap files into the prompt context, large files can increase cost and truncate important instructions [R9]. The compiler must run context-budget linting on every template.

- Warn when any single injected workspace file approaches the OpenClaw per-file limit.
- Warn when combined auto-injected files and required skill instructions exceed the tenant's configured context budget.
- Reject templates that rely on enormous AGENTS.md files instead of modular skills.
- Prefer on-demand memory retrieval for large reference corpora instead of auto-injected prose.

9. Workflow, task contract, and collaboration specification

Packs become valuable when many employees can coordinate on repeatable work. Collaboration must happen through typed tasks and structured artifacts, not improvised message passing. The orchestrator remains the owner of business state; employees produce proposals and outputs.

9.1 Workflow template fields

Field	Purpose
workflow.id / title	Stable identifier and display name.
triggerContracts[]	Signals, events, or manual actions that can start the workflow.
taskGraph	Ordered or conditional tasks, dependencies, and assignment rules.

Field	Purpose
stateMachine	Allowed lifecycle states and transitions for work items and tasks.
artifactBindings	Which artifact schemas are produced, read, or updated at each step.
approvalCheckpoints	Where human approval is required before progress may continue.
slaTargets	Timers, due dates, escalation windows, and retry budgets.
lockPolicy	Concurrency rules for write-backs or shared records.

9.2 Collaboration rules

- Each task has exactly one primary assignee at any time.
- Employees may request subtasks, but only the orchestrator creates official task records and assigns ownership.
- Shared business context lives in artifacts and task fields; private scratch work remains inside employee sessions and workspaces.
- Two employees must not write back to the same external record concurrently without a lock or optimistic concurrency check.
- Employees may propose a state transition, but the orchestrator commits it only after contract validation and approval checks pass.
- External outbound messages, published files, ticket mutations, or irreversible writes route through an approval policy unless explicitly classified low-risk.

9.3 Canonical collaboration patterns

Pattern	When to use it	Example
Supervisor + workers	A coordinating employee assigns specialized subtasks to others.	Reporting coordinator assigns evidence gathering to a document-control worker and summary writing to an analysis worker.
Pipeline handoff	Work always follows the same ordered sequence.	Issue intake → spec → builder orchestration → eval → release.
Parallel gather, single writer	Several employees research in parallel but only one is allowed to publish.	Construction monitoring employees collect evidence, while a final reporting employee prepares the customer-facing update.
Human gate	High-impact step requires decision by a person or approval group.	External write-back to Jira, Procore, or customer email.

10. Connector and capability specification

Packs should not be hard-wired to a single vendor when the business capability is broader. The specification therefore separates normalized capabilities from provider-specific connector bindings.

10.1 Capability model

Capability class	Examples	Notes
Read	files.read, tickets.read, messages.read, records.read	Low-risk by default but still subject to data-access policy.
Draft	artifacts.write, summaries.generate, forms.prepare	May create internal artifacts without changing external systems.
Write-back	tickets.update, project_records.update, messages.send	Typically approval-gated unless reversible and explicitly low-risk.
Control	cron.manage, sessions.spawn, gateway.control	Denied by default for any employee exposed to outside input.
Execution	repo.pull_request.create, workflow.run, report.publish	May combine internal and external effects; always policy-reviewed.

10.2 Connector binding rules

- A pack may declare required capabilities and supported providers, but provider installation refs and secret refs are overlay values.
- The connector profile must document which operations are dry-run safe, reversible, approval-required, or forbidden in unattended mode.
- The compiler should fail if a template requires a capability that no selected provider mapping can satisfy.
- Capability names must stay stable even when provider APIs change; provider-specific churn belongs inside connector adapters, not inside templates.

11. Policy, approvals, and risk controls

Every pack has an implicit risk posture. The specification makes that posture explicit through policy profiles so that customers can understand what each employee may do before anything is deployed.

11.1 Risk tiers

Tier	Meaning	Default policy
L0	Internal analysis only; no external side effects.	May run automatically if connector and data policies allow read access.
L1	Internal artifact creation or reversible draft actions.	Auto-run allowed; final external publish still requires escalation path.

Tier	Meaning	Default policy
L2	External write-back or outbound communications with meaningful customer impact.	Approval required unless an org-approved exception profile exists.
L3	Irreversible, high-impact, privileged, or break-glass actions.	Human-only or explicitly break-glass with named approver and audit reason.

11.2 Required policy profile fields

Field	Why it matters
approvalMode	Defines whether a task is auto, approval-required, or human-only.
approvalGroupRef	Links high-impact actions to a human owner or team.
allowedTools / deniedTools	Prevents templates from inheriting unsafe runtime defaults.
channelPolicy	Restricts where the employee may receive requests and where it may send output.
writeBackPolicy	Specifies which systems and record types may be mutated.
auditRequirements	Defines mandatory evidence attached to decisions and write-backs.
sandboxProfileRef	Ensures runtime isolation matches the employee's risk tier.
quietHours / rateLimits	Prevents automation spam or after-hours behavior.

Non-negotiable V1 rule. Any capability that can publish outward, mutate a customer system, or bypass a safety control must appear in a named policy profile. Silent inherited privilege is not allowed.

12. Evals, tests, and release gates

A pack is only reusable if it can be tested repeatedly. V1 therefore treats evals as first-class pack content, not optional add-ons. Every certified pack release must ship the eval suites needed to prove role behavior, workflow correctness, policy enforcement, and connector safety.

12.1 Eval suite categories

Eval category	What it checks	Typical fixture
Instruction compliance	Whether the employee follows role rules and output structure.	Representative task prompts and expected artifact schema.

Eval category	What it checks	Typical fixture
Tool choice	Whether the employee selects the right tool or abstains correctly.	Mocked tool availability and desired call pattern.
Workflow routing	Whether tasks route to the correct employee and state transitions.	Signal payloads across happy and edge paths.
Approval behavior	Whether high-risk actions pause for approval.	Simulated outbound send or write-back request.
Connector contract	Whether provider-specific payloads map cleanly.	Dry-run API stubs and fixture responses.
Regression	Whether known failures stay fixed.	Captured failure cases from previous deployments.
Cost and latency	Whether the pack stays within budget envelope.	Synthetic production-like run volumes.

12.2 Publish gates

- Schema validation: all manifests, schemas, and references resolve.
- Dependency validation: no missing pack deps, no unsupported version ranges, no dependency cycles.
- Prompt lint: workspace and skill content stays within configured context budgets.
- Compile dry-run: pack compiles into a bundle without unresolved capabilities or illegal overlays.
- Connector tests: required provider mappings pass dry-run contract checks.
- Policy scan: unsafe tools or channels are rejected for the target risk tier.
- Eval threshold: every mandatory suite meets minimum score before publish.
- Signing and integrity: release manifest, digests, and trusted publisher signature are recorded.

13. Compilation, overlay application, deployment, and rollback

Publishing a pack does not place it in production. Deployment is a second step that applies overlays, validates environment readiness, and generates a runtime bundle for one tenant trust boundary. The bundle is immutable and tied to the exact pack version, overlay values, and runtime adapter used to build it.

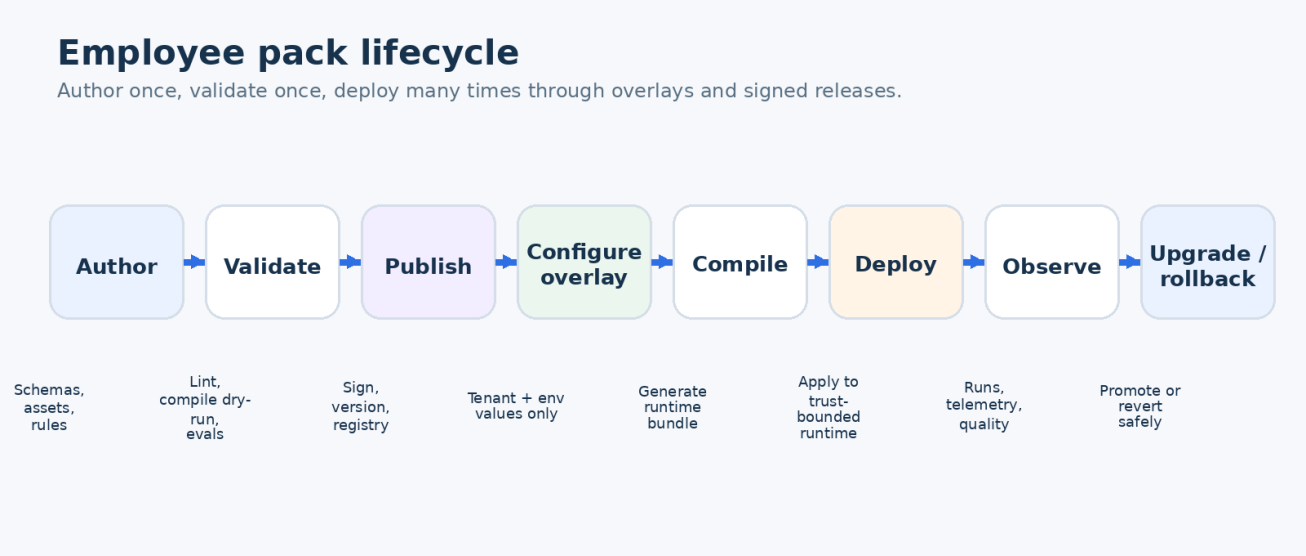

Figure 3. Author once, validate once, and deploy many times using controlled overlays and signed releases.

13.1 Compiler output

- Deployment manifest with pack version, dependency graph, overlay values, and artifact digests.
- Agent workspace trees and prompt assets for each employee instance.
- Skill and hook install plan, including curated package refs and pinned versions.
- OpenClaw config fragments: agents list, bindings, tool policy, sandbox config, heartbeat or cron setup.
- Connector binding manifest with secret references and provider installation refs.
- Rollback bundle pointing to the previous healthy deployment revision.

13.2 Upgrade rules

- Patch releases may not break schemas, exported identifiers, or allowed overlay paths.
- Minor releases may add templates, workflows, fields, or safer default policies, but must preserve existing exported contracts.
- Major releases are required for breaking exported schemas, removed templates, or changed approval semantics.
- The deployment service must show a semantic diff between old and new pack versions before promotion.
- Rollback must restore the previous runtime bundle and connector/policy mapping without editing pack source.

14. Registry, dependency, and compatibility model

V1 uses a curated private registry. Public marketplace installation is out of scope because production packs may influence tool exposure, hook behavior, and external system writes. The registry is therefore a trust surface, not just a convenience feature.

Concern	V1 rule
Registry visibility	Private registry only; customer tenants can consume approved packs but not install arbitrary public ones.

Concern	V1 rule
Dependency resolution	Semantic-version ranges allowed, but certified deployments must resolve to exact versions and digests.
Publisher trust	Only trusted publishers may release production-certified packs.
Compatibility enforcement	Pack install fails if platform, runtime adapter, or OpenClaw range is unsupported.
Deprecation	Deprecated exports remain installable for a defined support window with clear migration notes.

15. V1 authoring and release checklist

The following checklist is the minimum bar for a production-ready pack. The intent is to turn pack quality into a repeatable discipline instead of a manual review ritual.

Area	Acceptance condition
Identity	Pack id, version, compatibility ranges, and exports are present and valid.
Structure	Required directories exist; no stray executable content or secrets in source tree.
Templates	Each exported employee template declares objective, contracts, tool profile, sandbox posture, and eval suite refs.
Workflows	State machine, task ownership, and artifact outputs are explicit.
Policies	Risk tier, approval rules, channel restrictions, and write-back policy are defined.
Connectors	Required capabilities are satisfiable by supported providers.
Context budgets	Injected files and required skills pass prompt lint.
Evals	Mandatory smoke and regression suites pass release thresholds.
Compilation	Dry-run bundle succeeds against at least one supported target environment.
Rollback	Previous version remains deployable and upgrade notes are recorded.

16. Reference pack blueprints for Ruh.ai

To keep the pack system horizontal, Ruh.ai should ship a small set of reference packs that demonstrate reuse across industries. Two strong early examples are shown below because they reflect the company's current market direction and the broader digital-employee platform thesis.

Reference pack	Included employees	Core workflows	Why it matters
Construction Project Operations Pack	Project intake coordinator, document-control assistant, reporting assistant, action-item chaser.	Daily reporting, RFI/submittal prep, progress summary, follow-up management.	Proves cross-customer reuse in construction without becoming a one-off solution for one subcontractor or one sensor vendor.
AI BuildOps Pack	Intake & triage, spec writer, builder orchestrator, QA & evals, release assistant.	Issue-to-PR, regression-eval loop, release preparation, incident follow-up.	Shows the same pack system can serve AI product companies and validates the platform beyond construction.

Appendix A. Example pack-to-runtime compilation map

```
template: buildops.triage
compiles_to:
  agentId: tenantA-buildops-triage
  workspace:
    root: /srv/ruh/runtime/tenantA/prod/buildops-triage
    files:
      AGENTS.md: templates/triage/prompts/AGENTS.md
      SOUL.md: templates/common/SOUL.md
      TOOLS.md: overlays/prod/TOOLS.generated.md
      HEARTBEAT.md: templates/triage/prompts/HEARTBEAT.md
  skills:
    - workspace/skills/evidence_summarization
    - workspace/skills/issue_classification
  openclaw:
    bindings:
      - slack:eng-ops
    tools:
      allow: [read, web_fetch, message, sessions_history]
      deny: [gateway, cron, sessions_spawn, sessions_send]
    sandbox:
      mode: all
      scope: agent
      workspaceAccess: none
```

Appendix B. Source references

The references below anchor the OpenClaw assumptions used in this document. They should be revisited whenever the runtime adapter or pack compiler is updated.

Ref	Source	Link
R1	OpenClaw agents	https://docs.openclaw.ai/cli/agents
R2	OpenClaw skills	https://docs.openclaw.ai/tools/skills
R3	OpenClaw hooks	https://docs.openclaw.ai/automation/hooks
R4	OpenClaw tools	https://docs.openclaw.ai/tools
R5	OpenClaw agent workspace	https://docs.openclaw.ai/concepts/agent-workspace
R6	OpenClaw security model	https://docs.openclaw.ai/gateway/security
R7	OpenClaw sandboxing	https://docs.openclaw.ai/gateway/sandboxing
R8	OpenClaw configuration reference	https://docs.openclaw.ai/gateway/configuration-reference
R9	OpenClaw token use and context budgets	https://docs.openclaw.ai/reference/token-use
R10	OpenClaw heartbeat guidance	https://docs.openclaw.ai/gateway/heartbeat